

LAPORAN PRAKTIKUM DAN TUGAS 3
SISTEM OPERASI



Dosen pengampu :
Triawan Adi Cahyono M.Kom

Disusun Oleh:
Nama : Putri Suci Rahmadhani
Nim : 2410651008
Tanggal : 24 Oktober 2025

PRODI TEKNIK INFORMATIKA FAKULTAS TEKNIK
UNIVERSITAS MUHAMMADIYAH JEMBER

2025

1.1. TUJUAN PERCOBAAN DAN TUGAS

1. Memahami penerapan dasar operasi aritmatika dengan Bahasa c (input, proses, output).
2. Mempelajari cara pemrosesan file teks menggunakan thread untuk membandingkan kinerja single thread dan multi thread.
3. Mempelajari sinkronisasi data menggunakan buffer dengan batas maksimum agar proses antar thread berjalan aman dan efisien.

1.2.DASAR TEORI

1. Operasi Aritmatika Dasar

Adalah proses matematika sederhana seperti penjumlahan, pengurangan, perkalian, dan pembagian. Dalam pemrograman C, operasi ini digunakan untuk mengolah data numerik dengan operator aritmatika.

2. Thread (Multithreading)

Thread adalah unit terkecil dari suatu proses yang dapat dijalankan secara paralel. Dengan thread, program dapat membagi pekerjaan menjadi beberapa bagian untuk dieksekusi secara bersamaan, meningkatkan kecepatan pemrosesan data.

3. File Processing

Proses membaca, menulis, atau memanipulasi data yang tersimpan di dalam file. Pada tugas ini digunakan untuk membaca teks dari file eksternal dan menghitung jumlah kata secara efisien.

4. Buffer dan Sinkronisasi

Buffer adalah ruang penyimpanan sementara yang digunakan untuk menampung data sebelum diproses. Sinkronisasi diperlukan agar thread tidak saling bertabrakan ketika mengakses buffer secara bersamaan. Biasanya diatur menggunakan mutex atau semaphore.

1.3.PERCOBAAN

1. Percobaan 1 membuat thread sederhana

a. Codingan

```

GNU nano 4.8
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>
// Fungsi yang akan dijalankan oleh thread
void* cetak_pesan(void* arg) {
    char* pesan = (char*) arg;
    for(int i = 1; i <= 5; i++) {
        printf("%s - hitungan ke-%d\n", pesan, i);
        sleep(1); // Tidur 1 detik
    }
    return NULL;
}
int main() {
    pthread_t thread1, thread2;
    printf("=== Program Dimulai ===\n\n");
    // Membuat 2 thread
    pthread_create(&thread1, NULL, cetak_pesan, "Thread 1");
    pthread_create(&thread2, NULL, cetak_pesan, "Thread 2");
    // Menunggu kedua thread selesai
    pthread_join(thread1, NULL);
    pthread_join(thread2, NULL);
    printf("\n=== Program Selesai ===\n");
    return 0;
}

```

b. Hasil output program

```

madha_27@DESKTOP-S58N8I3:~$ nano thread_sederhana.c
madha_27@DESKTOP-S58N8I3:~$ gcc thread_sederhana.c -o thread_sederhana -pthread
madha_27@DESKTOP-S58N8I3:~$ ./thread_sederhana
=== Program Dimulai ===

Thread 1 - hitungan ke-1
Thread 2 - hitungan ke-1
Thread 1 - hitungan ke-2
Thread 2 - hitungan ke-2
Thread 1 - hitungan ke-3
Thread 2 - hitungan ke-3
Thread 1 - hitungan ke-4
Thread 2 - hitungan ke-4
Thread 2 - hitungan ke-5
Thread 1 - hitungan ke-5

=== Program Selesai ===
madha_27@DESKTOP-S58N8I3:~$ |

```

2. Percobaan 2 thread dengan shared memory

a. Codingan

```

GNU nano 4.8
#include <stdio.h>
#include <pthread.h>
int counter = 0; // Variabel yang dibagi bersama
void* tambah_counter(void* arg) {
    for(int i = 0; i < 100000; i++) {
        counter++; // Menambah counter
    }
    return NULL;
}
int main() {
    pthread_t thread1, thread2;

    printf("Counter awal: %d\n", counter);

    // Buat 2 thread yang menambah counter
    pthread_create(&thread1, NULL, tambah_counter, NULL);
    pthread_create(&thread2, NULL, tambah_counter, NULL);

    // Tunggu kedua thread selesai
    pthread_join(thread1, NULL);
    pthread_join(thread2, NULL);

    printf("Counter akhir: %d\n", counter);
    printf("Harusnya: 200000\n");

    return 0;
}

```

b. Hasil output program

```

madha_27@DESKTOP-S58N8I3:~$ nano thread_shared.c
madha_27@DESKTOP-S58N8I3:~$ gcc thread_shared.c -o thread_shared -pthread
madha_27@DESKTOP-S58N8I3:~$ ./thread_shared
Counter awal: 0
Counter akhir: 132244
Harusnya: 200000
madha_27@DESKTOP-S58N8I3:~$ |

```

3. Percobaan 3 mengatasi race condition dengan mutex

a. Codingan

```

GNU nano 4.8
#include <stdio.h>
#include <pthread.h>
int counter = 0;
pthread_mutex_t lock; // Variabel mutex (kunci)
void* tambah_counter(void* arg) {
    for(int i = 0; i < 100000; i++) {
        pthread_mutex_lock(&lock); // Kunci pintu
        counter++; // Area aman
        pthread_mutex_unlock(&lock); // Buka kunci
    }
    return NULL;
}
int main() {
    pthread_t thread1, thread2;

    // Inisialisasi mutex
    pthread_mutex_init(&lock, NULL);

    printf("Counter awal: %d\n", counter);

    pthread_create(&thread1, NULL, tambah_counter, NULL);
    pthread_create(&thread2, NULL, tambah_counter, NULL);

    pthread_join(thread1, NULL);
    pthread_join(thread2, NULL);

    printf("Counter akhir: %d\n", counter);
    printf("Harusnya: 200000\n");

    // Hapus mutex
    pthread_mutex_destroy(&lock);

    return 0;
}

```

b. Hasil output program

```
madha_27@DESKTOP-S58N8I3:~$ nano thread_mutex.c
madha_27@DESKTOP-S58N8I3:~$ gcc thread_mutex.c -o thread_mutex -pthread
madha_27@DESKTOP-S58N8I3:~$ ./thread_mutex
Counter awal: 0
Counter akhir: 200000
Harusnya: 200000
madha_27@DESKTOP-S58N8I3:~$ |
```

4. Percobaan 4 thread pool sederhana

a. Codingan

```
GNU nano 4.8
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>
#define JUMLAH_THREAD 3
#define JUMLAH_TUGAS 10
void* kerjakan_tugas(void* arg) {
    int id = *(int*) arg;

    printf("Thread %d: Mulai mengerjakan tugas\n", id);
    sleep(2); // Simulasi pekerjaan
    printf("Thread %d: Selesai mengerjakan tugas\n", id);

    return NULL;
}
int main() {
    pthread_t threads[JUMLAH_THREAD];
    int thread_ids[JUMLAH_THREAD];
    printf("=== Thread Pool dengan %d threads ===\n\n", JUMLAH_THREAD);

    // Buat pool of threads
    for(int i = 0; i < JUMLAH_THREAD; i++) {
        thread_ids[i] = i + 1;
        pthread_create(&threads[i], NULL, kerjakan_tugas, &thread_ids[i]);
    }

    // Tunggu semua thread selesai
    for(int i = 0; i < JUMLAH_THREAD; i++) {
        pthread_join(threads[i], NULL);
    }

    printf("\n=== Semua tugas selesai ===\n");
    return 0;
}
```

b. Hasil output program

```
madha_27@DESKTOP-S58N8I3:~$ nano thread_pool.c
madha_27@DESKTOP-S58N8I3:~$ gcc thread_pool.c -o thread_pool -pthread
madha_27@DESKTOP-S58N8I3:~$ ./thread_pool
=== Thread Pool dengan 3 threads ===

Thread 1: Mulai mengerjakan tugas
Thread 2: Mulai mengerjakan tugas
Thread 3: Mulai mengerjakan tugas
Thread 1: Selesai mengerjakan tugas
Thread 3: Selesai mengerjakan tugas
Thread 2: Selesai mengerjakan tugas

=== Semua tugas selesai ===
madha_27@DESKTOP-S58N8I3:~$ |
```

5. Percobaan 5 data parallelism

a. Codingan

```

GNU nano 4.8
#include <stdio.h>
#include <pthread.h>
#define UKURAN_ARRAY 1000
#define JUMLAH_THREAD 4
int array[UKURAN_ARRAY];
int jumlah_parsial[JUMLAH_THREAD];
typedef struct {
    int id;
    int mulai;
    int akhir;
} ThreadData;
void* hitung_jumlah(void* arg) {
    ThreadData* data = (ThreadData*) arg;
    int sum = 0;

    for(int i = data->mulai; i < data->akhir; i++) {
        sum += array[i];
    }

    jumlah_parsial[data->id] = sum;
    printf("Thread %d: Menghitung indeks %d-%d, hasil = %d\n",
        data->id, data->mulai, data->akhir-1, sum);

    return NULL;
}
int main() {
    pthread_t threads[JUMLAH_THREAD];
    ThreadData thread_data[JUMLAH_THREAD];

    // Isi array dengan angka 1-1000
    for(int i = 0; i < UKURAN_ARRAY; i++) {
        array[i] = i + 1;
    }

    printf("=== Data Parallelism: Menjumlahkan array ===\n\n");

    int bagian = UKURAN_ARRAY / JUMLAH_THREAD;

    // Buat thread untuk memproses bagian array
    for(int i = 0; i < JUMLAH_THREAD; i++) {
        thread_data[i].id = i;
        thread_data[i].mulai = i * bagian;
        thread_data[i].akhir = (i + 1) * bagian;

        pthread_create(&threads[i], NULL, hitung_jumlah, &thread_data[i]);
    }

    // Tunggu semua thread selesai
    for(int i = 0; i < JUMLAH_THREAD; i++) {
        pthread_join(threads[i], NULL);
    }

    // Jumlahkan hasil dari semua thread
    int total = 0;
    for(int i = 0; i < JUMLAH_THREAD; i++) {
        total += jumlah_parsial[i];
    }

    printf("\n=== Total keseluruhan: %d ===\n", total);
    printf("Verifikasi: 1+2+...+1000 = %d\n", (UKURAN_ARRAY * (UKURAN_ARRAY + 1)) / 2);

    return 0;
}

```

b. Hasil output program

```
madha_27@DESKTOP-S58N8I3:~$ nano data_parallel.c
madha_27@DESKTOP-S58N8I3:~$ gcc data_parallel.c -o data_parallel -pthread
madha_27@DESKTOP-S58N8I3:~$ ./data_parallel
=== Data Parallelism: Menjumlahkan array ===

Thread 0: Menghitung indeks 0-249, hasil = 31375
Thread 1: Menghitung indeks 250-499, hasil = 93875
Thread 2: Menghitung indeks 500-749, hasil = 156375
Thread 3: Menghitung indeks 750-999, hasil = 218875

=== Total keseluruhan: 500500 ===
Verifikasi: 1+2+...+1000 = 500500
madha_27@DESKTOP-S58N8I3:~$ |
```

6. Percobaan 6 mendeteksi jumlah CPU core

a. Codengan

```
GNU nano 4.8
#include <stdio.h>
#include <unistd.h>
#include <pthread.h>
void* tugas_berat(void* arg) {
    int id = *(int*) arg;
    long hasil = 0;

    printf("Thread %d: Memulai komputasi...\n", id);

    // Simulasi komputasi berat
    for(long i = 0; i < 100000000; i++) {
        hasil += i;
    }

    printf("Thread %d: Selesai!\n", id);
    return NULL;
}

int main() {
    // Deteksi jumlah CPU core
    int jumlah_core = sysconf(_SC_NPROCESSORS_ONLN);

    printf("=== Informasi Sistem ===\n");
    printf("Jumlah CPU core: %d\n\n", jumlah_core);

    pthread_t threads[jumlah_core];
    int thread_ids[jumlah_core];

    printf("Membuat %d thread (sesuai jumlah core)...\n\n", jumlah_core);

    // Buat thread sebanyak jumlah core
    for(int i = 0; i < jumlah_core; i++) {
        thread_ids[i] = i + 1;
        pthread_create(&threads[i], NULL, tugas_berat, &thread_ids[i]);
    }

    // Tunggu semua selesai
    for(int i = 0; i < jumlah_core; i++) {
        pthread_join(threads[i], NULL);
    }

    printf("\n=== Semua thread selesai ===\n");

    return 0;
}
```

b. Hasil output program

```

madha_27@DESKTOP-S58N8I3:~$ nano cpu_cores.c
madha_27@DESKTOP-S58N8I3:~$ gcc cpu_cores.c -o cpu_cores -pthread
madha_27@DESKTOP-S58N8I3:~$ ./cpu_cores
=== Informasi Sistem ===
Jumlah CPU core: 8

Membuat 8 thread (sesuai jumlah core)...

Thread 1: Memulai komputasi...
Thread 2: Memulai komputasi...
Thread 3: Memulai komputasi...
Thread 4: Memulai komputasi...
Thread 5: Memulai komputasi...
Thread 6: Memulai komputasi...
Thread 7: Memulai komputasi...
Thread 8: Memulai komputasi...
Thread 3: Selesai!
Thread 5: Selesai!
Thread 1: Selesai!
Thread 6: Selesai!
Thread 2: Selesai!
Thread 4: Selesai!
Thread 7: Selesai!
Thread 8: Selesai!

=== Semua thread selesai ===
madha_27@DESKTOP-S58N8I3:~$ |

```

1.4.TUGAS AKHIR

1. Tugas 1

a. Codingan

```

GNU nano 4.8
#include <stdio.h>

int main() {
    float a, b;
    char op;

    printf("=== TUGAS 1 - OPERASI ARITMATIKA DASAR ===\n");
    printf("Masukkan angka pertama : ");
    scanf("%f", &a);
    printf("Masukkan operator (+ - * /) : ");
    scanf(" %c", &op);
    printf("Masukkan angka kedua : ");
    scanf("%f", &b);

    printf("\n=== HASIL PERHITUNGAN ===\n");
    switch (op) {
        case '+':
            printf("%.0f + %.0f = %.0f\n", a, b, a + b);
            break;
        case '-':
            printf("%.0f - %.0f = %.0f\n", a, b, a - b);
            break;
        case '*':
            printf("%.0f * %.0f = %.0f\n", a, b, a * b);
            break;
        case '/':
            if (b != 0)
                printf("%.0f / %.0f = %.2f\n", a, b, a / b);
            else
                printf("Error: Pembagian dengan nol tidak valid.\n");
            break;
        default:
            printf("Operator tidak dikenal.\n");
    }

    printf("=====\n");
    return 0;
}

```

b. Penjelasan algoritma

1. #include <stdio.h>

= Baris ini digunakan untuk memasukkan library standar input-output di bahasa C. Fungsinya agar kita bisa memakai fungsi seperti printf() dan scanf().

2. int main() {

= Ini adalah fungsi utama tempat eksekusi program dimulai.

3. float a, b;

= Mendeklarasikan dua variabel bertipe float bernama a dan b untuk menyimpan dua angka yang akan dihitung (karena bisa berupa desimal).

4. char op;

= Mendeklarasikan variabel op bertipe char untuk menyimpan operator aritmatika (+, -, *, atau /) yang dimasukkan pengguna.

5. printf("=== TUGAS 1 — OPERASI ARITMATIKA DASAR ===\n");

= Menampilkan judul program di layar.

6. printf("Masukkan angka pertama : ");

= Meminta pengguna untuk memasukkan angka pertama.

scanf("%f", &a);

= Membaca input pengguna dan menyimpannya ke variabel a.

7. printf("Masukkan operator (+ - * /) : ");

= Menampilkan instruksi agar pengguna memilih operator.

scanf(" %c", &op);

= Membaca satu karakter operator dari input. Ada spasi sebelum %c agar bisa mengabaikan karakter newline dari input sebelumnya.

8. printf("Masukkan angka kedua : ");

= Meminta pengguna untuk memasukkan angka kedua.

scanf("%f", &b);

= Menyimpan nilai yang dimasukkan ke variabel b.

9. printf("\n=== HASIL PERHITUNGAN ===\n");

= Menampilkan header bagian hasil perhitungan.

10. switch (op) { ... }

= Struktur percabangan yang digunakan untuk memilih operasi berdasarkan operator yang dimasukkan pengguna (op).

11. case '+': printf("%.0f + %.0f = %.0f\n", a, b, a + b); break;

= Jika operator +, maka program akan menjumlahkan a dan b, lalu menampilkan hasilnya tanpa angka desimal (%.0f).

12. case '-': printf("%.0f - %.0f = %.0f\n", a, b, a - b); break;

= Jika operator -, maka dilakukan pengurangan.

13. case '*': printf("%.0f * %.0f = %.0f\n", a, b, a * b); break;

= Jika operator *, maka dilakukan perkalian.

14. case '/':

= Jika operator /, maka program akan memeriksa dulu apakah pembagian valid.

if (b != 0) = Mengecek apakah b bukan nol.

Jika iya, tampilkan hasil bagi dengan dua angka di belakang koma (%.2f).

Jika b sama dengan nol, tampilkan pesan error:

"Error: Pembagian dengan nol tidak valid."

15. default:

= Jika operator yang dimasukkan bukan salah satu dari + - * /, maka tampilkan pesan "Operator tidak dikenal."

16. printf("=====\n");

= Menampilkan garis penutup hasil agar tampilan rapi.

17. return 0;

= Menandakan program selesai dengan status sukses.

18. }

= Penutup fungsi main().

c. Hasil output program

```
madha_27@DESKTOP-S58N8I3:~$ nano tugas1.c
madha_27@DESKTOP-S58N8I3:~$ gcc tugas1.c -o tugas1
madha_27@DESKTOP-S58N8I3:~$ ./tugas1
=== TUGAS 1 – OPERASI ARITMATIKA DASAR ===
Masukkan angka pertama : 27
Masukkan operator (+ - * /) : +
Masukkan angka kedua   : 4

=== HASIL PERHITUNGAN ===
27 + 4 = 31
=====
madha_27@DESKTOP-S58N8I3:~$ |
```

2. Tugas 2

a. Codingan

```

#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <string.h>
#include <ctype.h>
#include <time.h>

typedef struct {
    char* text;
    long start;
    long end;
    int count;
} ThreadData;

void* count_words(void* arg) {
    ThreadData* data = (ThreadData*)arg;
    int in_word = 0, count = 0;

    for (long i = data->start; i < data->end; i++) {
        if (data->text[i] == ' ' || data->text[i] == '\n' || data->text[i] == '\t')
            in_word = 0;
        else if (!in_word) {
            in_word = 1;
            count++;
        }
    }

    data->count = count;
    pthread_exit(NULL);
}

int count_words_single(char* text, long length) {
    int in_word = 0, count = 0;

    for (long i = 0; i < length; i++) {
        if (text[i] == ' ' || text[i] == '\n' || text[i] == '\t')
            in_word = 0;
        else if (!in_word) {
            in_word = 1;
            count++;
        }
    }

    return count;
}

int main(int argc, char* argv[]) {
    if (argc < 2) {
        printf("Gunakan: %s <nama_file>\n", argv[0]);
        return 1;
    }

    FILE* file = fopen(argv[1], "r");
    if (!file) {
        perror("Gagal membuka file");
        return 1;
    }

    fseek(file, 0, SEEK_END);
    long length = ftell(file);
    fseek(file, 0, SEEK_SET);

    char* text = malloc(length + 1);
    fread(text, 1, length, file);
    text[length] = '\0';
    fclose(file);

    long mid = length / 2;
    while (mid < length && isalpha(text[mid])) mid++;
    pthread_t t1, t2;
    ThreadData d1 = { text, 0, mid, 0 };
    ThreadData d2 = { text, mid, length, 0 };

    clock_t start_mt = clock();
    pthread_create(&t1, NULL, count_words, &d1);
    pthread_create(&t2, NULL, count_words, &d2);
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    int total_mt = d1.count + d2.count;
    double time_mt = (double)(clock() - start_mt) / CLOCKS_PER_SEC;

    clock_t start_st = clock();
    int total_st = count_words_single(text, length);
    double time_st = (double)(clock() - start_st) / CLOCKS_PER_SEC;

    printf("\n=== HASIL FILE PROCESSING ===\n");
    printf("Nama file      : madha\n");
    printf("Jumlah kata     : %d\n", total_st);
    printf("2 Thread time   : %.6f detik\n", time_mt);
    printf("1 Thread time   : %.6f detik\n", time_st);
    printf("Perbandingan    : %s\n",
        total_mt == total_st ? "✅ Hasil sama" : "⚠️ Hasil berbeda");

    free(text);
    return 0;
}

```

b. Penjelasan algoritma

```

#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <string.h>
#include <ctype.h>

```

```
#include <time.h>
```

Bagian ini memanggil library yang dibutuhkan:

stdio.h → untuk input/output seperti printf, fopen

stdlib.h → untuk fungsi umum seperti malloc, free

pthread.h → untuk membuat thread

string.h → untuk operasi string

ctype.h → untuk fungsi karakter seperti isalpha

time.h → untuk menghitung waktu eksekusi

```
typedef struct { char* text; long start; long end; int count; } ThreadData;
```

Struct ThreadData digunakan untuk menyimpan data yang dikirim ke masing-masing thread:

text → teks dari file

start → posisi awal thread membaca

end → posisi akhir thread membaca

count → jumlah kata yang ditemukan thread itu

```
void* count_words(void* arg) { ThreadData* data = (ThreadData*)arg;
int in_word = 0, count = 0; for (long i = data->start; i < data->end; i++) {
if (data->text[i] == ' ' || data->text[i] == '\n' || data->text[i] == '\t') in_word
= 0; else if (!in_word) { in_word = 1; count++; } } data->count = count;
pthread_exit(NULL); }
```

Fungsi ini dijalankan oleh setiap thread:

Membaca bagian teks dari start sampai end

Menghitung jumlah kata dengan cara mengenali spasi, newline, atau tab sebagai pemisah

in_word menandakan apakah sedang di dalam kata atau tidak

Hasil jumlah kata disimpan di data->count

Thread selesai dengan pthread_exit()

```
int count_words_single(char* text, long length) { int in_word = 0, count =
0; for (long i = 0; i < length; i++) { if (text[i] == ' ' || text[i] == '\n' || text[i]
== '\t') in_word = 0; else if (!in_word) { in_word = 1; count++; } } return
count; }
```

Fungsi ini menghitung jumlah kata tanpa thread (single thread).
Sama seperti fungsi sebelumnya, tapi hanya berjalan di satu proses.

```
int main(int argc, char* argv[]) { if (argc < 2) { printf("Gunakan: %s  
<nama_file>\n", argv[0]); return 1; }
```

Bagian ini mengecek apakah pengguna sudah memasukkan nama file di argumen saat menjalankan program.

Contoh menjalankan: ./program teks.txt

Jika belum ada nama file, tampil pesan kesalahan.

```
FILE* file = fopen(argv[1], "r"); if (!file) { perror("Gagal membuka file");  
return 1; }
```

Membuka file teks yang diberikan lewat argumen.

Jika gagal membuka, tampil pesan error.

```
fseek(file, 0, SEEK_END); long length = ftell(file); fseek(file, 0,  
SEEK_SET); char* text = malloc(length + 1); fread(text, 1, length, file);  
text[length] = '\0'; fclose(file);
```

Bagian ini membaca isi file:

fseek dan ftell → mencari panjang file

malloc → mengalokasikan memori untuk teks

fread → membaca isi file

\0 → menandai akhir teks (string terminator)

fclose → menutup file

```
long mid = length / 2; while (mid < length && isalpha(text[mid])) mid++;
```

Menentukan titik tengah teks (mid) untuk dibagi ke dua thread.

Jika mid berhenti di tengah kata, maka loop ini memajukan posisi agar tidak membelah kata di tengah.

```
pthread_t t1, t2; ThreadData d1 = { text, 0, mid, 0 }; ThreadData d2 = {  
text, mid, length, 0 };
```

Membuat dua variabel thread (t1, t2) dan dua data (d1, d2):

d1 memproses bagian awal teks

d2 memproses bagian akhir teks

```
clock_t start_mt = clock(); pthread_create(&t1, NULL, count_words,
&d1); pthread_create(&t2, NULL, count_words, &d2); pthread_join(t1,
NULL); pthread_join(t2, NULL); int total_mt = d1.count + d2.count;
double time_mt = (double)(clock() - start_mt) / CLOCKS_PER_SEC;
```

Bagian ini adalah proses multithreading:

pthread_create → memulai dua thread

pthread_join → menunggu kedua thread selesai

total_mt → hasil gabungan dari dua thread

time_mt → menghitung waktu eksekusi multithread

```
clock_t start_st = clock(); int total_st = count_words_single(text, length);
double time_st = (double)(clock() - start_st) / CLOCKS_PER_SEC;
```

Bagian ini menghitung kata tanpa thread (single-thread) dan mengukur waktu eksekusi.

```
printf("\n=== HASIL FILE PROCESSING ===\n"); printf("Nama file :
madha\n"); printf("Jumlah kata : %d\n", total_st); printf("2 Thread time :
%.6f detik\n", time_mt); printf("1 Thread time : %.6f detik\n", time_st);
printf("Perbandingan : %s\n", total_mt == total_st ? "✅ Hasil sama" :
"⚠️ Hasil berbeda");
```

Menampilkan hasil:

Nama file (contoh: "madha")

Jumlah kata

Waktu proses dua thread dan satu thread

Perbandingan apakah hasil keduanya sama

```
free(text); return 0; }
```

Membersihkan memori (free) dan mengakhiri program.

c. Hasil output program

```

madha_27@DESKTOP-S58N8I3:~$ nano file_processing.c
madha_27@DESKTOP-S58N8I3:~$ nano teks.txt
madha_27@DESKTOP-S58N8I3:~$ gcc file_processing.c -o file_processing -lpthread
madha_27@DESKTOP-S58N8I3:~$ ./file_processing teks.txt

=== HASIL FILE PROCESSING ===
Nama file      : madha
Jumlah kata    : 13
2 Thread time : 0.001442 detik
1 Thread time : 0.000002 detik
Perbandingan  : ✅ Hasil sama
madha_27@DESKTOP-S58N8I3:~$ |

```

Nano teks.txt untuk isi dari data yang ingin dihitung.

3. Tugas 3

a. Codingan

```

#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>

#define BUFFER_SIZE 10 // kapasitas maksimal buffer

int buffer[BUFFER_SIZE];
int count = 0; // jumlah item saat ini di buffer
int in = 0; // posisi producer menambah item
int out = 0; // posisi consumer mengambil item

pthread_mutex_t mutex;
pthread_cond_t not_full;
pthread_cond_t not_empty;

// fungsi untuk menampilkan isi buffer
void tampilkan_buffer() {
    printf("Isi buffer (%d/%d): [", count, BUFFER_SIZE);
    for (int i = 0; i < count; i++) {
        printf("%d", buffer[(out + i) % BUFFER_SIZE]);
        if (i < count - 1) printf(", ");
    }
    printf("]\n");
}

// fungsi producer
void* producer(void* arg) {
    for (int i = 1; i <= 10; i++) {
        int item = rand() % 100; // angka acak 0-99

        pthread_mutex_lock(&mutex);
        while (count == BUFFER_SIZE) {
            printf(" Buffer penuh! Producer menunggu sampai ada ruang kosong...\n");
            pthread_cond_wait(&not_full, &mutex);
        }

        buffer[in] = item;
        in = (in + 1) % BUFFER_SIZE;
        count++;

        printf("✅ Producer membuat item ke-%d: %d\n", i, item);
        tampilkan_buffer();

        if (count == BUFFER_SIZE) {
            printf("❗ Buffer sudah penuh (%d item)! Producer berhenti sementara.\n", BUFFER_SIZE);
        }

        pthread_cond_signal(&not_empty);
        pthread_mutex_unlock(&mutex);

        usleep(150000); // jeda 0.15 detik
    }

    pthread_exit(NULL);
}

// fungsi consumer
void* consumer(void* arg) {
    int i = 1;
    while (1) {
        pthread_mutex_lock(&mutex);
        while (count == 0) {
            printf("❗ Buffer kosong! Consumer menunggu item...\n");
            pthread_cond_wait(&not_empty, &mutex);
        }

        int item = buffer[out];
        out = (out + 1) % BUFFER_SIZE;
        count--;

        printf(" Consumer mengambil item ke-%d: %d\n", i++, item);
        tampilkan_buffer();

        if (count == 0) {
            printf("⚠ Buffer sekarang kosong! Consumer berhenti sementara.\n");
        }

        pthread_cond_signal(&not_full);
        pthread_mutex_unlock(&mutex);
    }
}

```

```

        usleep(200000); // jeda 0.2 detik

        if (i > 10 && count == 0)
            break; // berhenti setelah 10 item diproses
    }

    pthread_exit(NULL);
}

int main() {
    pthread_t prod, cons;

    srand(time(NULL));
    pthread_mutex_init(&mutex, NULL);
    pthread_cond_init(&not_full, NULL);
    pthread_cond_init(&not_empty, NULL);

    printf("=== SIMULASI PRODUCER-CONSUMER (BUFFER MAKS 10) ===\n\n");

    pthread_create(&prod, NULL, producer, NULL);
    pthread_create(&cons, NULL, consumer, NULL);

    pthread_join(prod, NULL);
    pthread_join(cons, NULL);

    printf("\n✅ Semua proses selesai. Buffer sudah kosong kembali.\n");

    pthread_mutex_destroy(&mutex);
    pthread_cond_destroy(&not_full);
    pthread_cond_destroy(&not_empty);

    return 0;
}

```

b. Penjelasan algoritma

```

#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>

```

Library yang digunakan:

stdio.h → untuk input/output (printf)

stdlib.h → untuk fungsi umum seperti rand()

pthread.h → untuk membuat dan mengatur thread

unistd.h → untuk fungsi usleep() agar memberi jeda waktu

Deklarasi Variabel Global

```

#define BUFFER_SIZE 10
int buffer[BUFFER_SIZE];
int count = 0;
int in = 0;
int out = 0;
pthread_mutex_t mutex;
pthread_cond_t not_full;
pthread_cond_t not_empty;

```

Penjelasan:

BUFFER_SIZE → kapasitas maksimal buffer (10 item)

buffer[] → tempat penyimpanan sementara data dari producer

count → menghitung jumlah item yang sedang ada di buffer

in → posisi index untuk producer menambahkan item

out → posisi index untuk consumer mengambil item

pthread_mutex_t → digunakan untuk mengunci (lock) proses agar tidak bentrok antar-thread

pthread_cond_t → kondisi untuk sinkronisasi antara producer dan consumer (not_full = buffer belum penuh, not_empty = buffer tidak kosong)

Fungsi untuk Menampilkan Isi Buffer

```
void tampilkan_buffer() {  
    printf("Isi buffer (%d/%d): [", count, BUFFER_SIZE);  
    for (int i = 0; i < count; i++) {  
        printf("%d", buffer[(out + i) % BUFFER_SIZE]);  
        if (i < count - 1) printf(", ");  
    }  
    printf("]\n");  
}
```

Fungsi ini menampilkan isi buffer saat ini dan menunjukkan berapa item yang ada dari total kapasitas.

Fungsi Producer

```
void* producer(void* arg) {  
    for (int i = 1; i <= 10; i++) {  
        int item = rand() % 100;  
        pthread_mutex_lock(&mutex);  
        while (count == BUFFER_SIZE) {  
            printf("□ Buffer penuh! Producer menunggu sampai ada ruang  
kosong...\n");  
            pthread_cond_wait(&not_full, &mutex);  
        }  
        buffer[in] = item;  
        in = (in + 1) % BUFFER_SIZE;  
        count++;  
        printf("☑ Producer membuat item ke-%d: %d\n", i, item);  
        tampilkan_buffer();  
    }
```

```

        if (count == BUFFER_SIZE) {
            printf("⊙ Buffer sudah penuh (%d item)! Producer berhenti
sementara.\n\n", BUFFER_SIZE);
        }
        pthread_cond_signal(&not_empty);
        pthread_mutex_unlock(&mutex);
        usleep(150000);
    }
    pthread_exit(NULL);
}

```

Penjelasan langkah-langkah:

1. Producer membuat 10 item acak (0–99).
2. Mengunci mutex agar hanya satu thread yang bisa mengakses buffer.
3. Jika buffer penuh, producer menunggu sampai ada ruang kosong.
4. Menyimpan item ke buffer.
5. Menggeser posisi in dengan operasi modulo.
6. Menambah jumlah item di buffer.
7. Menampilkan item yang dimasukkan.
8. Memberi sinyal ke consumer bahwa buffer tidak kosong.
9. Melepas kunci mutex dan jeda 0.15 detik.

Fungsi Consumer

```

void* consumer(void* arg) {
    int i = 1;
    while (1) {
        pthread_mutex_lock(&mutex);
        while (count == 0) {
            printf("● Buffer kosong! Consumer menunggu item...\n");
            pthread_cond_wait(&not_empty, &mutex);
        }
        int item = buffer[out];
        out = (out + 1) % BUFFER_SIZE;
        count--;
        printf("□ Consumer mengambil item ke-%d: %d\n", i++, item);
        tampilkan_buffer();
        if (count == 0) {
            printf("△ Buffer sekarang kosong! Consumer berhenti
sementara.\n\n");
        }
    }
}

```

```

        pthread_cond_signal(&not_full);
        pthread_mutex_unlock(&mutex);
        usleep(200000);
        if (i > 10 && count == 0) break;
    }
    pthread_exit(NULL);
}

```

Penjelasan langkah-langkah:

1. Consumer mengambil item dari buffer.
2. Jika buffer kosong, consumer menunggu sampai producer menambah item.
3. Mengambil item dari posisi out dan mengurangi count.
4. Menampilkan isi buffer terbaru.
5. Jika buffer kosong, menampilkan pesan berhenti sementara.
6. Memberi sinyal ke producer bahwa buffer tidak penuh lagi.
7. Melepas mutex, jeda 0.2 detik, lalu berhenti jika semua item sudah diambil.

Fungsi Main

```

int main() {
    pthread_t prod, cons;
    srand(time(NULL));
    pthread_mutex_init(&mutex, NULL);
    pthread_cond_init(&not_full, NULL);
    pthread_cond_init(&not_empty, NULL);
    printf("=== SIMULASI PRODUCER-CONSUMER (BUFFER MAKS
10) ===\n\n");
    pthread_create(&prod, NULL, producer, NULL);
    pthread_create(&cons, NULL, consumer, NULL);
    pthread_join(prod, NULL);
    pthread_join(cons, NULL);
    printf("\n✅ Semua proses selesai. Buffer sudah kosong kembali.\n");
    pthread_mutex_destroy(&mutex);
    pthread_cond_destroy(&not_full);
    pthread_cond_destroy(&not_empty);
    return 0;
}

```

Penjelasan:

1. Membuat dua thread: prod (producer) dan cons (consumer).
2. Inisialisasi mutex dan kondisi.
3. Menjalankan kedua thread secara bersamaan.
4. pthread_join menunggu kedua thread selesai.
5. Menampilkan pesan bahwa buffer sudah kosong.
6. Menghapus (destroy) mutex dan kondisi sebelum program selesai.

c. Hasil output program

1.5.KESIMPULAN

Dari tugas 1 mengajarkan dasar logika aritmatika dan struktur program c menggunakan input output.

Sedangkan ditugas 2 memperkenalkan konsep multithreading dan menunjukan perbedaan waktu eksekusi antara proses single thread dan multi thread.

Berbeda lagi dengan tugas 3 memperdalam konsep sinkronisasi thread dengan buffer maksimal 10, yang melatih pemahaman terhadap komunikasi antar proses secara aman dan efisien.